

DOCUMENTAÇÃO GERAL · MVP

Diário APLV

Acompanhamento simples, registro seguro.

13 de 13 módulos concluídos

diario-aplv.netlify.app

React 19 · TypeScript · Supabase

PWA · pt-BR · mobile-first

SUMÁRIO

- 01 O que é

- 02 O objetivo

- 03 O percurso de uso

- 04 O que se registra

- 05 A escada do leite

- 06 Timeline e relatório

- 07 Arquitetura

- 08 Modelo de dados

- 09 Segurança

- 10 O que foi construído

- 11 Fora do MVP

- 12 Rodar e publicar

DEFINIÇÃO

O que é o Diário APLV

O Diário APLV é uma aplicação web (com comportamento de PWA instalável) para famílias que acompanham uma criança em **reintrodução de leite e derivados** — o processo conhecido como *escada do leite*, na suspeita ou no diagnóstico de alergia à proteína do leite de vaca.

Ele faz três coisas, e só três: **registra** o que aconteceu com o mínimo de esforço, **organiza** tudo numa linha do tempo única, e **imprime** um relatório para a consulta.

LIMITE DO PRODUTO

O Diário APLV registra fatos e organiza. Ele **não** estabelece diagnóstico de APLV, não classifica gravidade, não sugere conduta e não recomenda avançar ou parar uma etapa. Nenhuma tela conclui, interpreta ou recomenda — a interpretação é sempre do profissional de saúde responsável.

Esse limite não é um aviso legal colado no rodapé: é uma regra de arquitetura. Não existe regra automática de avanço de etapa em lugar nenhum do código, e o relatório apresenta intervalos de tempo entre registros sem jamais nomeá-los como causa.

OBJETIVO

Por que ele existe

Quem acompanha uma criança em reintrodução precisa responder a uma pergunta simples na consulta: *o que aconteceu, e quando?* Na prática, o registro compete com o colo, a mamada e a noite mal dormida. Se registrar custa caro, não se registra — e a consulta vira memória aproximada.

Daí a meta objetiva do produto: **registrar exposição, sintoma ou fralda em menos de 30 segundos**, com o mínimo de digitação possível. Todas as decisões de interface derivam dela.

ACESSO

1 toque da Home

Qualquer ação principal está a no máximo um toque da tela inicial. São 8 atalhos, sem menu escondido.

ENTRADA

Digitação é exceção

Botão, chip ou seletor antes de teclado. O único campo livre obrigatório do fluxo rápido é o alimento da exposição.

TEMPO

Horário já preenchido

Todo evento nasce com o horário atual, sempre editável para quem registra depois.

CONFIANÇA

Salvou, apareceu

O formulário só limpa depois da confirmação do banco. Nada de registro que some.

A mensagem fixa da tela de sintomas

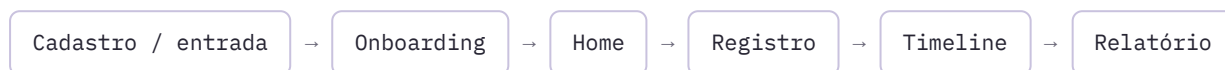
Registre apenas o que realmente observar. Não é necessário procurar sintomas.

Ela existe porque um diário de sintomas induz vigilância: quem procura, acha. O texto é constante na tela, não um aviso que aparece uma vez.

FLUXO

O percurso de uso

Uma família entra uma vez, cadastra a criança e o acompanhamento, e a partir daí o app é só a Home e os oito atalhos.



O **onboarding** acontece uma vez: nome e nascimento da criança, alimentação atual, motivo do acompanhamento, profissional de saúde (opcional) e data de início. Ele cria criança, acompanhamento e primeira etapa numa única transação no banco. Enquanto não existe acompanhamento ativo, o app leva de volta para o onboarding; enquanto não existe sessão, leva para o login.

Os oito atalhos da Home

ATALHO	O QUE FAZ	ROTA
Exposição	O que foi consumido, em que quantidade	/app/exposicao
Sintomas	Marcar sintomas por toque, com intensidade	/app/sintomas
Sem sintomas	Registrar que está tudo bem	/app/sem-sintomas
Fralda	Sangue, muco e consistência	/app/fralda
Observação	Anotação livre	/app/observacao
Timeline	Tudo em ordem cronológica	/app/timeline
Etapas	Avançar, repetir, retornar na escada	/app/etapas
Relatório	Documento para levar à consulta	/app/relatorio

CONTEÚDO

O que se registra

Exposição

O alimento (campo livre, com sugestão das exposições recentes para reaproveitar o que já foi digitado), a quantidade em quatro toques — **pequena · habitual · maior que o habitual · não sei** — o horário e uma observação opcional.

Sintomas

Catálogo fechado de **21 sintomas em 4 grupos**: gastrointestinais e fezes (10), pele (4), respiratórios e estado geral (6) e outros (1). O toque não é no sintoma e depois na intensidade: é o **toque na intensidade que**

seleciona o sintoma — Leve, Moderada ou Intensa — o que corta um passo de cada marcação.

Seis sintomas são marcados como **sinais de alerta** (sangue nas fezes, urticária, inchaço, dificuldade para respirar, palidez importante, sonolência ou prostração). Ao marcá-los, a tela exibe uma orientação de cuidado — *procure o profissional de saúde responsável para avaliar este sinal* — que é encaminhamento, não diagnóstico nem classificação de gravidade.

Um evento de sintomas pode ser **vinculado a uma exposição**. O vínculo é opcional e é o que alimenta a tabela de temporalidade do relatório.

Sem sintomas

Um registro de um toque, com valor próprio: a ausência de sintomas em um dia é informação clínica, e sem ela o diário só mostra os dias ruins. No banco é o mesmo evento de sintomas, com `no_symptoms = true` e nenhum item.

Fralda

Três seletores por toque: **sangue** (não · traços · visível), **muco** (não · pouco · moderado · muito) e **consistência** (habitual · líquida · pastosa · ressecada · não sei). Foto ficou fora do MVP.

Observação

Texto livre com horário, para o que não cabe em nenhum dos formulários acima.

PROTOCOLO

A escada do leite

O acompanhamento tem cinco etapas nomeadas, da forma menos alergênica do leite (assado, com a proteína mais modificada pelo calor) até o leite in natura.

1	Preparação assada	
2	Derivado aquecido	
3	Queijo	
4	logurte	
5	Leite	

São três ações possíveis — **avancar**, **repetir** e **retornar** — e todas passam por uma confirmação com o mesmo texto: *avance apenas se estiver seguindo o plano definido pela equipe assistente*. Não existe avanço automático: a decisão é humana e clínica, e o app apenas a registra.

O histórico é **imutável**. Mudar de etapa nunca sobrescreve nada: fecha o período corrente com data de fim e desfecho (avanzou · repetiu · retornou · pausou) e insere uma linha nova. Uma família pode passar pela etapa 3 três vezes, e as três aparecem. Além disso, **todo evento guarda a etapa vigente no momento do registro** — é o que permite ao relatório agrupar por etapa mesmo depois de a família ter avançado.

Timeline e relatório

As cinco origens do diário — exposições, eventos de sintomas, fraldas, observações e mudanças de etapa — são buscadas em paralelo e unidas **no frontend**, ordenadas por horário do acontecimento. Não existe tabela nem view de timeline no banco: a união teria que acontecer no cliente de qualquer forma para o relatório, e a decisão reduz superfície no banco.

O relatório

O documento levado à consulta é impresso a partir do navegador e tem cinco seções:

- **Criança e acompanhamento** — nome, nascimento, alimentação, período coberto, etapa atual, motivo e profissional.
- **Registros no período** — contagem de exposições, sintomas, registros sem sintomas, fraldas e observações.
- **Resumo por etapa** — dias vividos, número de períodos e contagens por etapa da escada.
- **Temporalidade** — para cada sintoma vinculado a uma exposição, o intervalo entre os dois registros, rotulado como *distância no tempo*, nunca como causa.
- **Registros em ordem** — a timeline completa do período.

Abre e fecha com o mesmo aviso: *este relatório organiza os dados registrados pela família; não estabelece diagnóstico de APLV e deve ser interpretado pelo profissional de saúde responsável.*

Arquitetura

CAMADA	ESCOLHA
UI	React 19 + TypeScript 6, modo estrito, sem <code>any</code>
Build	Vite 8 + vite-plugin-pwa
Backend	Supabase — PostgreSQL, Auth e Row Level Security
Roteamento	React Router 7
Formulários	React Hook Form + Zod
Datas	date-fns, locale pt-BR
Ícones	Lucide React
Estilo	CSS Modules + design tokens em custom properties
Qualidade	oxlint + Prettier + <code>tsc -b</code>
Hospedagem	Netlify, deploy contínuo a partir da <code>main</code>

Um fluxo de dados, sem atalhos



A regra é única e não tem exceção: `supabase.from(...)` dentro de `components/` é proibido. A página conhece a rota e orquestra hooks; o hook carrega o estado (`idle` · `saving` · `success` · `error`); o service é a única camada que fala com o Supabase.

Componentes em Atomic Design

Átomos (Button, Chip, Input, Select, Textarea, Badge, Loading) → **moléculas** (Card, ChipGroup, SeveritySelector, TimelineItem, Toast, StageProgress, Alert, EmptyState) → **organismos** (os formulários, TimelineList, ReportDocument, Modal) → **templates** (Auth, App, Protocol). Um catálogo vivo de todos eles fica em `/dev`, carregado sob demanda e apenas em desenvolvimento — em produção o chunk nunca é pedido.

PWA

Service worker com atualização por aviso, instalável na tela inicial, orientação retrato, ícones e manifest próprios. O cache guarda **somente o app**: nada de respostas do Supabase. Dado de saúde não fica guardado no aparelho, e uma resposta velha de banco é pior do que um aviso honesto de “sem conexão”.

BANCO

Modelo de dados

Nove tabelas, todas descendentes do usuário autenticado:

```
auth.users
├── profiles (1:1)
│   ├── children (1:N)
│   │   ├── protocols (1:N) — o "acompanhamento"
│   │   │   ├── stage_history (1:N)
│   │   │   ├── exposures (1:N)
│   │   │   ├── symptom_events (1:N) — symptom_event_items (1:N)
│   │   │   ├── diaper_records (1:N)
│   │   └── notes (1:N)
```

Toda tabela filha carrega `user_id` denormalizado — RLS simples e rápida, sem JOIN recursivo em cada policy. Todo evento carrega `occurred_at` e `stage`.

Enums e catálogos

Seis enums no banco, um para cada lista curta e fechada definida pelo produto: `protocol_status`, `stage_outcome`, `exposure_amount`, `diaper_blood`, `diaper_mucus`, `diaper_consistency`. O catálogo de sintomas e o de alimentação ficam em `src/constants/` como texto — assim crescem sem migration, e um código já usado em produção nunca é removido.

Migrations e funções

Oito migrations versionadas em `supabase/migrations/`, aplicadas em ordem e nunca editadas depois de aplicadas — mudança vira migration nova. Os tipos TypeScript do banco são gerados a partir dele, não escritos à mão.

Três operações que precisam ser atômicas vivem como funções no banco: `create_onboarding` (criança + acompanhamento + primeira etapa), `create_symptom_event` (evento + itens) e `change_stage` (fecha o período corrente e abre o próximo). Mais o trigger `handle_new_user`, que cria o perfil no cadastro.

PRIORIDADE N° 1

Segurança dos dados

São dados de saúde de uma criança. A ordem de prioridade do projeto é explícita:



- **RLS ligada nas 9 tabelas**, nunca desativada para ganhar velocidade de desenvolvimento. São 36 policies apoiadas em 4 funções de posse.
- **Dupla checagem de posse**. Cada policy verifica que a linha é minha e que o registro-pai também é meu. Só `user_id = auth.uid()` deixaria alguém que descobrisse um `protocol_id` alheio inserir registros dentro do acompanhamento de outra família — a chave estrangeira não verifica dono. A falha foi reproduzida em Postgres e corrigida com as funções `owns_*`.
- **Só a publishable key vai para o frontend**. A service role key nunca entra no repositório nem no painel de deploy: qualquer variável com prefixo `VITE_` é embutida no bundle e chega ao navegador de quem usa o app.
- **Nada indexado**. O site vai com `X-Robots-Tag: noindex, nofollow`, referrer restrito, `nosniff`, `X-Frame-Options: DENY`, HSTS e câmera, microfone e geolocalização desligados por Permissions-Policy.
- **Nenhum dado pessoal além do necessário**. Fotos, por exemplo, ficaram deliberadamente fora do MVP.

O que foi construído

O plano tem 13 módulos lineares. Cada um entrega algo funcional, e seu critério de aceite é a porta do seguinte. A numeração aqui é a sequência real de construção — o M1 vem antes do design justamente porque segurança precede acabamento.

#	MÓDULO	O QUE ENTREGOU	STATUS
M0	Fundação	Vite, TypeScript estrito, aliases, lint, formatação, estrutura de pastas	Concluído
M1	Supabase, schema e RLS	6 enums, 9 tabelas, 36 policies, trigger de perfil — migrations aplicadas e validadas	Concluído
M2	Auth e rotas protegidas	Cadastro, entrada, sessão persistente, rotas públicas e protegidas	Concluído
M3	Onboarding	Criança + acompanhamento + primeira etapa em uma transação	Concluído
M4	Shell e Dashboard	Home com os 8 atalhos, etapa atual, contexto do acompanhamento ativo	Concluído
M5	Exposição	Formulário rápido com quantidade por toque e sugestão de recentes	Concluído
M6	Sintomas rápidos	21 sintomas, intensidade por toque, sinais de alerta, vínculo com exposição, “sem sintomas”	Concluído
M7	Timeline essencial	União das cinco origens no cliente, em ordem cronológica MARCO DE USO REAL	Concluído
M8	Fralda e observações	Três seletores da fralda e anotação livre	Concluído
M9	Etapas da escada	Avançar, repetir e retornar com confirmação; histórico imutável	Concluído
M10	Relatório e impressão	Documento de consulta com resumo por etapa, temporalidade e timeline	Concluído
M11	Design System	Tokens, componentes, animações discretas e catálogo em <code>/dev</code>	Concluído
M12	PWA e QA final	Manifest, service worker, instalação, aviso de nova versão e estado offline	Concluído

O código dos 13 módulos está escrito e no ar. O que continua desmarcado nos documentos de módulo são os **critérios de aceite que exigem conferência manual em navegador** — testes de toque nos breakpoints de 375 a 1440 px, instalação do PWA, impressão do relatório. São a única coisa entre o estado de hoje e o MVP liberado para uso.

Fora do MVP, e conscientemente

O diário, hoje, só cria registros. Ficaram deliberadamente para depois:

- Corrigir e apagar registro — é o primeiro item a entrar depois do MVP, porque a especificação original o pedia.
- Redefinir senha.
- Pausar ou encerrar o acompanhamento.
- Editar os dados da criança.
- Acompanhar mais de uma criança.
- Fotos de fralda (e, com elas, o Supabase Storage).
- Estratégia offline-first com IndexedDB.

Pendências conhecidas

- O catálogo de **alimentação atual** (5 opções) foi para produção como provisório: a especificação pede o campo mas não define as opções, e elas não passaram por validação clínica.
- A especificação original batiza o produto de **Lactra**; a interface hoje se chama **Diário APLV**. É uma decisão de marca em aberto, não um erro de implementação.

OPERAÇÃO

Rodar e publicar

```
git clone git@github.com:Joamarcellodev/Diario_APLV.git
cd Diario_APLV
npm install
cp .env.example .env.local # chaves do seu projeto Supabase
npm run dev
```

O Vite só expõe variáveis com prefixo `VITE_`: são `VITE_SUPABASE_URL` e `VITE_SUPABASE_PUBLISHABLE_KEY`. O banco sobe com `npx supabase db push`, e os tipos saem dele com `npx supabase gen types typescript --linked`.

SCRIPT	O QUE FAZ
<code>npm run dev</code>	Servidor de desenvolvimento
<code>npm run build</code>	Typecheck e build de produção
<code>npm run preview</code>	Serve o build
<code>npm run typecheck</code>	<code>tsc -b --noEmit</code>
<code>npm run lint</code>	oxlint
<code>npm run format</code>	Prettier

A publicação é contínua: todo push na `main` dispara build no Netlify e publica. Build, publish, redirect de SPA e cabeçalhos de segurança vêm todos do `netlify.toml` — o painel não precisa ser tocado, exceto

para cadastrar as duas variáveis de ambiente antes do primeiro build e liberar a URL em Authentication → URL Configuration no Supabase.

Diário APLV · acompanhamento simples, registro seguro. Documentação geral do MVP — o plano de construção completo, módulo a módulo, está em [docs/](#) no repositório, e [docs/05-roadmap.md](#) é a autoridade em qualquer divergência.